



Smart Contract Security Audit Report

Gearbox Midas Integration Audit

1. Contents

1.	Contents	2
2.	General Information	3
2.1.	Introduction	3
2.2.	Scope of Work	3
2.3.	Threat Model.....	3
2.4.	Weakness Scoring	4
2.5.	Disclaimer	4
3.	Summary.....	5
3.1.	Suggestions	5
4.	General Recommendations	6
4.1.	Security Process Improvement	6
5.	Findings.....	7
5.1.	Gateway's tokens can be drained	7
5.2.	Single redemption vault architecture breaks multi-vault Midas strategies	7
5.3.	First redemption request becomes permanently unclaimable	8
5.4.	Single concurrent redemption request limit	9
5.5.	Fee-on-transfer tokens may cause permanent withdrawal lock.....	10
5.6.	Unused function parameter in depositInstant.....	11
6.	Appendix.....	13
6.1.	About us	13

2. General Information

This report contains information about the results of the security audit of the (hereafter referred to as “Customer”) smart contracts, conducted by [Decurity](#) in the period from 2025-10-07 to 2025-10-09.

2.1. Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2. Scope of Work

The audit scope included the contracts in the following repository: <https://github.com/Gearbox-protocol/integrations-v3/commits/midas/>. Initial review was done for the commit dd64e3573ef467f8daee4379a9e66915595b8489 and the re-testing was done for the commit e53ff92202fe2a16d12ec28617d4356666ef95dc.

The following contracts have been tested:

- contracts/adapters/midas/MidasIssuanceVaultAdapter.sol
- contracts/adapters/midas/MidasRedemptionVaultAdapter.sol
- contracts/helpers/midas/MidasRedemptionVaultGateway.sol
- contracts/helpers/midas/MidasRedemptionVaultPhantomToken.sol
- contracts/helpers/mellow/MellowWithdrawalPhantomToken.sol

2.3. Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- User,
- Protocol owner,
- Midas protocol.

2.4. Weakness Scoring

An expert evaluation scores the findings in this report, an impact of each vulnerability is calculated based on its ease of exploitation (based on the industry practice and our experience) and severity (for the considered threats).

2.5. Disclaimer

Due to the intrinsic nature of the software and vulnerabilities and the changing threat landscape, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided “as is” and is not a guarantee that the analyzed system does not contain any other security weaknesses or vulnerabilities. Furthermore, this report is not an endorsement of the Customer’s project, nor is it an investment advice.

That being said, Decurity exercises best effort to perform their contractual obligations and follow the industry methodologies to discover as many weaknesses as possible and maximize the audit coverage using the limited resources.

3. Summary

As a result of work we have discovered critical and medium security issues.

3.1. Suggestions

The table below contains the discovered issues, their risk level, and their status as of October 10, 2025.

Table. Discovered weaknesses

Issue	Contract	Risk Level	Status
Gateway's tokens can be drained	contracts/helpers/midas/MidasRedemptionVaultGateway.sol	Critical	Fixed
Single redemption vault architecture breaks multi-vault Midas strategies	contracts/helpers/midas/MidasRedemptionVaultGateway.sol	Medium	Fixed
First redemption request becomes permanently unclaimable	contracts/helpers/midas/MidasRedemptionVaultGateway.sol	Medium	Fixed
Single concurrent redemption request limit	contracts/helpers/midas/MidasRedemptionVaultGateway.sol	Low	Acknowledged
Fee-on-transfer tokens may cause permanent withdrawal lock	contracts/helpers/midas/MidasRedemptionVaultGateway.sol	Low	Acknowledged
Unused function parameter in depositInstant	/contracts/adapters/midas/MidasIssuanceVaultAdapter.sol	Info	Acknowledged

4. General Recommendations

This section contains general recommendations on how to improve overall security level.

The Findings section contains technical recommendations for each discovered issue.

4.1. Security Process Improvement

The following is a brief long-term action plan to mitigate further weaknesses and bring the product security to a higher level:

- Keep the whitepaper and documentation updated to make it consistent with the implementation and the intended use cases of the system,
- Perform regular audits for all the new contracts and updates,
- Ensure the secure off-chain storage and processing of the credentials (e.g. the privileged private keys),
- Launch a public bug bounty campaign for the contracts.

5. Findings

5.1. Gateway's tokens can be drained

Risk Level: Critical

Status: Fixed in the commit [89ab4fd](#)

Contracts:

- contracts/helpers/midas/MidasRedemptionVaultGateway.sol

Location:

- `redeemInstant()`

Description:

The `redeemInstant()` function transfers all of the gateway's tokens to the caller. However, the gateway also holds tokens from fulfilled redemption requests. As a result, anyone can call the function and withdraw tokens that belong to other users.

Remediation:

Consider sending only the balance delta that accrues after the `redeemInstant()` invocation.

5.2. Single redemption vault architecture breaks multi-vault Midas strategies

Risk Level: Medium

Status: Fixed in the commit [e53ff92](#)

Contracts:

- contracts/helpers/midas/MidasRedemptionVaultGateway.sol

Description:

The gateway has a single `midasRedemptionVault` address, used for both instant and standard redemptions. However, some Midas strategies deploy separate vaults for instant redemptions vs. standard (request-based) redemptions. For example, the `mTBILL` strategy contracts on Ethereum has 2 separate redemption vaults (see [Midas docs](#)). When a strategy uses separate redemption vaults, `MidasRedemptionVaultGateway` may initiate standard or instant requests through a wrong vault. In this case, either instant requests won't work or standard requests may remain unapproved.

Remediation:

Support separate vault addresses for instant vs. standard redemptions.

References:

<https://docs.midas.app/resources/smart-contracts-addresses>

5.3. First redemption request becomes permanently unclaimable

Risk Level: Medium

Status: Fixed in the commit [89ab4fd](#)

Contracts:

- `contracts/helpers/midas/MidasRedemptionVaultGateway.sol`

Location:

- `withdraw()`

Description:

The `withdraw()` function checks if a user has a pending redemption by verifying `pending.requestId == 0`. At the time of request creation, the `requestId` is set to the `currentRequestId` value taken from the `Midas RedemptionVault`. However, `Midas RedemptionVault` uses a counter starting at 0 for `currentRequestId`. In case a Gearbox user is the very first one to initiate the delayed redemption request from the Midas vault, the user will not be able to withdraw their funds from the gateway because of the zero `requestId` check.

Remediation:

Use a separate flag `exists` flag in the `PendingRedemption` structure to check if the request exists, or set the `requestId` to the `currentRequestId + 1` value.

5.4. Single concurrent redemption request limit

Risk Level: Low

Status: Acknowledged

Contracts:

- `contracts/helpers/midas/MidasRedemptionVaultGateway.sol`

Location:

- `requestRedeem()`

Description:

The gateway enforces a limit of one pending redemption request per user (line 54):

```
if (pendingRedemptions[msg.sender].requestId > 0) {  
    revert("MidasRedemptionVaultGateway: user has a pending  
redemption");  
}
```

This prevents users from creating a new redemption request while one is pending, even for different output tokens or if they want to add to their position.

Impact:

- Users must wait for request approval + full withdrawal before submitting another request
- During high request volumes or slow admin approval, users cannot adjust their positions
- Reduces capital efficiency and user experience
- Not a fund loss risk, but operational limitation

Remediation:

Support multiple concurrent redemption requests per user using a mapping from user to request

ID list:

```
struct PendingRedemption {
    uint256 requestId;
    uint256 timestamp;
    uint256 remainder;
    address tokenOut; // track token type per request
}
mapping(address => PendingRedemption[]) public pendingRedemptions;
function requestRedeem(address tokenOut, uint256 amountMTokenIn)
external nonReentrant {
    // No limit check, allow multiple pending requests
    uint256 requestId =
IMidasRedemptionVault(midasRedemptionVault).currentRequestId();
    IERC20(mToken).safeTransferFrom(msg.sender, address(this),
amountMTokenIn);
    IERC20(mToken).forceApprove(midasRedemptionVault, amountMTokenIn);
    IMidasRedemptionVault(midasRedemptionVault).redeemRequest(tokenOut,
amountMTokenIn);
    pendingRedemptions[msg.sender].push(
        PendingRedemption({
            requestId: requestId,
            timestamp: block.timestamp,
            remainder: 0,
            tokenOut: tokenOut
        })
    );
}
function withdraw(uint256 requestIndex, uint256 amount) external
nonReentrant {
    // Withdraw from specific request by index
    // Remove from array when fully withdrawn
}
```

Alternatively, implement a reasonable limit (e.g., 5 concurrent requests) to prevent array bloat while improving UX.

5.5. Fee-on-transfer tokens may cause permanent withdrawal lock

Risk Level: **Low**

Status: Fee-on-transfer tokens will not be supported in Gearbox via the token whitelist in the adapter. Also, it's not possible to track balance diff in this case.

Contracts:

- contracts/helpers/midas/MidasRedemptionVaultGateway.sol

Location:

- `withdraw()`

Description:

When calculating the remainder after partial withdrawal, the contract assumes the actual transferred amount equals the requested amount. However, fee-on-transfer tokens may transfer a smaller amount than requested. If such a token is supported as `tokenOut`, pending redemptions may reflect a bigger amount in `remainder`, which will be impossible to withdraw fully from the gateway, as the actual received amount on the gateway is less than calculated in `remainder`. In this case, a user will not be able to create a new redemption request, since `remainder` $\neq$ 0.

Remediation:

Track actual transferred amounts using balance differences.

5.6. Unused function parameter in depositInstant

Risk Level: Info

Status: Acknowledged

Contracts:

- /contracts/adapters/midas/MidasIssuanceVaultAdapter.sol

Location:

- `depositInstant()`

Description:

The `depositInstant()` function accepts a `bytes32` parameter (the fourth parameter) that is never used within the function body.

```
function depositInstant(address tokenIn, uint256 amountToken, uint256
minReceiveAmount, bytes32)
    external
    override
    creditFacadeOnly
    returns (bool)
```

Remediation:

Remove the unused `bytes32` parameter from the function signature.

6. Appendix

6.1. About us

The [Decurity](#) team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During the recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.